Programming Massively Parallel Processors

A Hands-on Approach

Fourth Edition

Chapter 4 – Compute architecture and scheduling

Exercise

Q1 Consider the following CUDA kernel and the corresponding host function that calls it:

```
01    __global__ void foo_kernel(int* a, int* b) {
02        unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
03        if(threadIdx.x < 40 || threadIdx.x >= 104) {
04            b[i] = a[i] + 1;
05        }
06        if(i%2 == 0) {
07            a[i] = b[i]*2;
08        }
09        for(unsigned int j = 0; j < 5 - (i%3); ++j) {
10            b[i] += j;
11        }
12    }
13    void foo(int* a_d, int* b_d) {
14        unsigned int N = 1024;
15        foo_kernel <<< (N + 128 - 1)/128, 128 >>>(a_d, b_d);
16    }
```

a. What is the number of warps per block?

128 / 32 = 4

b. What is the number of warps in the grid?

8 * 4 = 32

c. For the statement on line 04:

I. How many warps in the grid are active?

0-31 (O)

32-63 (O)

63-95 (X)

96-127 (O)

8 * 3 = 24

II. How many warps in the grid are divergent?

0-31 (X)

32-63 (O)

63-95 (X)

96-127 (O)

8 * 2 = 16

III. What is the SIMD efficiency (in %) of warp 0 of block 0?

100 %

IV. What is the SIMD efficiency (in %) of warp 1 of block 0?

8 / 32 = 25 %

v. What is the SIMD efficiency (in %) of warp 3 of block 0?

24 / 32 = 75 %

d. For the statement on line 07:

I. How many warps in the grid are active?

4 * 8 = 32

II. How many warps in the grid are divergent?

32

III. What is the SIMD efficiency (in %) of warp 0 of block 0?

50 %

e. For the loop on line 09:

I. How many iterations have no divergence?

3

II. How many iterations have divergence?

2

Q2. For a vector addition, assume that the vector length is 2000, each thread calculates one output element, and the thread block size is 512 threads. How many threads will be in the grid?

2048

Q3. For the previous question, how many warps do you expect to have divergence due to the boundary check on vector length?

0 – 31 (X)

32 – 63 (X)

.

.

1984 – 2015 (O)

2016 – 2047 (X)

1

Q4. Consider a hypothetical block with 8 threads executing a section of code before reaching a barrier. The threads require the following amount of time (in microseconds) to execute the sections: 2.0, 2.3, 3.0, 2.8, 2.4, 1.9, 2.6, and 2.9; they spend the rest of their time waiting for the barrier. What percentage of the threads' total execution time is spent waiting for the barrier?

1.0 + 0.7 + 0 + 0.2 + 0.6 + 1.1 + 0.4 + 0.1 = 4.1

4.1 / (3.0 * 8) = 17.08%

Q5. A CUDA programmer says that if they launch a kernel with only 32 threads in each block, they can leave out the __syncthreads() instruction wherever barrier synchronization is needed. Do you think this is a good idea? Explain.

Modern GPUs do not guarantee lockstep execution within a warp. Without __syncthreads(), race conditions may occur if threads read data before it is written. Synchronization should be used whenever cross-thread dependencies exist, and correctness should not rely on warp size.

Q6. If a CUDA device's SM can take up to 1536 threads and up to 4 thread blocks, which of the following block configurations would result in the most number of threads in the SM?

a. 128 threads per block
b. 256 threads per block
c. 512 threads per block
d. 1024 threads per block

C

Q7. Assume a device that allows up to 64 blocks per SM and 2048 threads per SM. Indicate which of the following assignments per SM are possible. In the cases in which it is possible, indicate the occupancy level.

a. 8 blocks with 128 threads each
b. 16 blocks with 64 threads each
c. 32 blocks with 32 threads each
d. 64 blocks with 32 threads each
e. 32 blocks with 64 threads each

A, B, C, D, E

A. (8 * 128) / 2048 = 50%

B. (16 * 64) / 2048 = 50%

C. (32 * 32) / 2048 = 50%

D. (64 * 32) / 2048 = 100%

E. (32 * 64) / 2048 = 100%

Q8. Consider a GPU with the following hardware limits: 2048 threads per SM, 32 blocks per SM, and 64K (65,536) registers per SM. For each of the following kernel characteristics, specify whether the kernel can achieve full occupancy. If not, specify the limiting factor.

a. The kernel uses 128 threads per block and 30 registers per thread.

b. The kernel uses 32 threads per block and 29 registers per thread.

c. The kernel uses 256 threads per block and 34 registers per thread.

A

A.

block = 2048 / 128 = 16 < 32 (O)

register = 30 × 2048 = 61,440 < 65,536 (O)

threads = 16 × 128 = 2048

occupancy = 2048 / 2048 = 100%

B.

block = 2048 / 32 = 64 > 32 (block limit : The max block number is 32)

threads = 32 × 32 = 1024

register = 29 × 1024 = 29,696 < 65,536 (OK)

occupancy = 1024 / 2048 = 50%

C.

block = 2048 / 256 = 8 < 32 (OK)

register = 34 × 2048 = 69,632 > 65,536

Available threads = 65,536 / 34 ≈ 1,927

Available blocks = 1,927 / 256 ≈ 7

Occupancy = 1,792 / 2,048 ≈ 87.5%

Q9. A student mentions that they were able to multiply two 1024 x 1024 matrices using a matrix multiplication kernel with 32 x 32 thread blocks. The student is using a CUDA device that allows up to 512 threads per block and up to 8 blocks per SM. The student further mentions that each thread in a thread block calculates one element of the result matrix. What would be your reaction and why?

It seems infeasible because the device specifies a maximum of 512 threads per block, while the user designed a 32×32 thread block, which contains 1024 threads. This exceeds the hardware limit, so the kernel configuration cannot be launched on that GPU.